AESC / NeuroField v2: Two-Speed Memory for Recurrent Language Models

Giridhari Karmakar
Independent researcher · Project AERIS_64
giriisdev@gmail.com
https://github.com/giridhari-code/AERIS_64

October 2026 · Technical research paper (preprint-style)

Abstract

We present **NeuroField (AESC v2)**, a recurrent sequence model that replaces full self-attention with (1) a **predictive dendrite** over a short causal window, (2) a **k-step neural field** that refines hidden state with local coupling and memory readout, (3) **top-k skill routing**, and (4) an explicit **two-speed memory**: fast weights updated by a surprise-gated, decay-consistent delta rule, and slow weights updated by surprise-weighted replay into a content-addressed ring. The shipped implementation uses a **parallel / looped** forward path: sequence-wide projections with a causal fast-memory scan across depth passes. We give complete forward equations, safety behaviour (write-gate freeze), training objectives, and an honest status report: the system trains, checkpoints, and serves, with causal leakage checks; language quality at scale remains limited by data and throughput, not by absence of a closed-form design. This paper is intended as a citable engineering–research record for further work.

Keywords: recurrent language models, fast weights, delta rule, neural field, skill routing, two-speed memory, AESC, AERIS

1. Introduction

Transformers dominate language modelling through stacked causal self-attention [Vaswani et al., 2017]. Their strength is global context mixing at every layer; their cost is quadratic attention and a largely uniform computational graph. A parallel research line revisits recurrence, fast weights, and modular computation: models that write temporary associations into a matrix memory, refine a continuous state, and allocate capacity through routing.

AESC / NeuroField follows that line. The design hypothesis is that next-token prediction can be supported by (a) local predictive structure in the input stream, (b) iterative field dynamics, (c) sparse skill specialists, and (d) separate timescales of memory—fast, high-plasticity weights for the current context and slow consolidated structure for longer span. The software product shell is named **AERIS_64**; this paper describes the scientific and engineering content of the architecture as implemented in the open repository.

Contributions: (i) a complete, implementable specification of the AESC v2 forward pass including the decay-consistent delta rule; (ii) a parallel/looped production path with explicit causality constraints; (iii) a safety mechanism that freezes plasticity via the write gate rather than destroying weights; (iv) an honest limitation section suitable for follow-on research.

2. Related work (brief)

Fast weights and delta-rule memories appear in classical and modern work on higher-order weights and linear attention-style associative memory. Neural ODEs and continuous attractor networks motivate iterative state updates; mixture-of-experts motivates top-k routing. Slow/fast memory echoes complementary learning systems. AESC is not claimed to supersede Transformers; it is an alternative inductive bias with different compute–memory tradeoffs (causal scan vs dense attention).

3. Architecture

3.1 High-level dataflow

```
token ids
  -> Embed                          X in R^{B x T x d}
  -> DendriteWindow                 D_win (causal, width W)
  -> for depth j = 1 .. k:
       Predict + delayed error      E
       Neuromodulator gate          G
       Skill router                 O
       Fast delta-rule scan         M, R   (causal over T)
```

```
      Neural field step           H
  -> Slow ring read               U
  -> Readout -> logits over vocab
```

3.2 Embedding

```
X_t = Embed(token_t)  in R^d
```

Optional weight tying binds the output projection to the embedding matrix.

3.3 Predictive dendrite

The dendrite aggregates a short causal window of embeddings and injects a delayed prediction error. This supplies local context without full attention.

```
d_win_t = tanh( sum_{j=0}^{W-1}  c_j * x_{t-j} + b )
d_t     = d_win_t + gamma * e_t

c_j, b, gamma are learnable; * denotes elementwise product where used.
Positions before the sequence are treated as zeros (or carried history at decode time).
```

3.4 Prediction, error, surprise

```
x_hat_{t+1} = f_p(h_t)                    # predictor network
e_{t+1}     = stopgrad(x_{t+1}) - x_hat_{t+1}
S_{t+1}     = stopgrad( mean_i |e_{t+1,i}| )
S_bar_t     = rho * S_bar_{t-1} + (1-rho) * S_t
S_tilde_t   = S_t / (S_bar_t + eps)       # relative surprise
```

Detaching the target embedding stabilises the error path so the predictor tracks the data stream rather than collapsing through a pure autoencoder shortcut.

3.5 Neuromodulatory write gate

```
g_t = sigmoid( a_s * (S_tilde_t - 1) + a_c * mean(|h_t|) + b + delta_t )
g_t = min(g_t, g_max)                     # optional hard cap

If memory_frozen (safety): g_t = 0
  => fast memory only decays; no plastic write.
```

3.6 Neural field

The field is a residual iterative map mixing previous state, dendrite input, local neuronal coupling, and memory readout:

```
h <- (1 - zeta) * h
     + zeta * LayerNorm(
           alpha * h
         + beta  * ( d_t + R(tanh(h)) + W_m * r_{t-1} )
       )

R is a shared depthwise-style 1D convolution over the neuron index (width 3).
In the parallel path, one field step (or a small fixed stack) runs per depth pass
for every time index with batched ops.
```

3.7 Skill router

```
pi    = softmax(W_r * h)                  # skill mixture
w     = softmax(top-k logits of pi)
w_STE = w + pi - stopgrad(pi)             # straight-through estimator
o_t   = sum_i w_STE_i * Op_i(h)           # specialist outputs
```

A load-balancing auxiliary loss encourages utilisation of multiple skills during training.

3.8 Fast memory: decay-consistent delta rule

Fast memory M is a matrix associating keys derived from past dendrite states to values derived from skill outputs. Updates follow a decay-consistent outer-product rule (Change 5 in the project notes): decay is applied before the residual write so that g_t = 0 implies pure decay.

```
k_t = normalize(W_k * d_{t-1})
v_t = W_v * o_t
Lambda = diag(sigmoid(theta))             # per-dim decay in (0,1)
```

```
M_dec  = Lambda * M_{t-1}
M_t    = M_dec + g_t * outer( k_t , (v_t - M_dec^T * k_t) )
M_t    <- M_t * min(1, M_max / frobenius(M_t))   # optional bound

Read:
q_t = normalize(W_q * d_t)
r_t = M_t^T * q_t

Proposition (gate off): if g_t = 0 then M_t = Lambda * M_{t-1}.
```

3.9 Slow memory and readout

```
u_t = SlowRead(d_t; ring of slots)        # content-addressed
z_t = W_o * LayerNorm( o_t + s * P_f(r_t) + P_s(u_t) )
logits_t = z_t                            # vocab scores

Replay (training sketch):
W_s <- (1-eta)*W_s + eta * sum_b (S^{(b)} / sum_b S^{(b)}) * M^{(b)}
```

3.10 Parallel / looped production path

Production code does not use a pure token-serial Python loop as the only path. It embeds all T positions, builds dendrite windows for all t, then runs k depth iterations. Inside each depth, most maps are batched over T; the fast-memory write is a **causal left-to-right scan**. A fresh M at each depth reduces leakage of future writes into early positions across depth. Autoregressive decoding must reproduce the same functional behaviour (parity tests).

```
X = Embed(tokens)                         # (B,T,d)
D_win = DendriteWindow(X)                 # causal
H = D_win
for j in 1..k:
    E = stopgrad(X) - shift(Predict(H))
    D = D_win + gamma * E
    O = RouterSkills(H)
    G = Gate(E, H)
    M, R = DeltaScan_causal(D, O, G)
    H = FieldStep(H, D, shift(R))
logits = Readout(O, R, U_slow)
```

4. Causality

AESC does not use a triangular attention mask. Causality is structural: (i) dendrite windows only see past embeddings; (ii) prediction error is one-step delayed via shift; (iii) fast memory writes in time order; (iv) field input uses shifted reads. Empirical check: perturbing x_{t+1} must not change logits at indices less than or equal to t.

5. Training objective

```
L = CE(logits, targets; label_smoothing)
  + lambda_err * mean(||E||^2)
  + lambda_bal * routing_balance_penalty
```

Optimiser: AdamW with gradient clipping. Schedules: linear warmup; optional cosine decay or Noam-style d_model^{-0.5} * min(step^{-0.5}, step * warmup^{-1.5}). Dropout and label smoothing (typical 0.1) follow standard LM practice. Distributed data parallel (torchrun) is supported; it does not change the mathematical forward path.

6. Safety observability

Under anomalous surprise patterns, a session flag memory_frozen forces g_t = 0. Fast memory then only multiplies by Lambda. This stops further writing without permanently zeroing parameters, and is intended as a lightweight runtime control for research deployments.

7. Implementation and empirical status

Reference code: Python / PyTorch package neurofield under AERIS_64 (modules: dendrite, field, memory, router, neuromodulator; training script; FastAPI serve; safetensors checkpoints). Small-scale runs on GPU show decreasing training loss and passing leakage checks when data is non-empty. Educational chat quality tracks data quality: empty or highly repeated corpora yield nonsensical generations even when loss is low on that corpus. Throughput is lower

than a Transformer of similar width at equal sequence length because of the causal memory scan and depth loop; practical levers are batch size, sequence length, reduced k, mixed precision, torch.compile, and multi-GPU DDP.

These results are **not** claimed as competitive benchmarks against large Transformer LMs. They establish implementability and basic correctness properties of the design.

8. Limitations and future work

1. **Data.** Pretraining and instruction data at meaningful scale are required before strong language behaviour is expected.
2. **Compute efficiency.** Causal delta scans limit tokens/sec versus dense attention kernels.
3. **Gate dynamics.** Collapse of g toward 0 disables plasticity; schedules and regularisers need study.
4. **Parity.** Train (full sequence) vs decode (stateful token steps) must stay matched under refactors.
5. **Evaluation.** Multi-seed tasks, long-context probes, and matched-FLOP Transformer baselines are pending.
6. **Theory.** Capacity and optimisation landscape of the decay-consistent rule under depth k are open.
7. **Safety.** Gate freeze is only a first control; broader policy and red-teaming remain future work.

9. Conclusion

NeuroField / AESC v2 specifies a complete two-speed-memory recurrent language model with predictive dendrite, neural field, skill routing, and a decay-consistent fast delta rule, plus a parallel/looped implementation path with explicit causality and a gate-based safety freeze. The design is implemented and trainable. Closing the gap to high-quality language modelling is primarily a matter of data, compute, and the research items listed above—not of missing core equations.

Author information

Giridhari Karmakar
Email: giriisdev@gmail.com
Project: AERIS_64 / NeuroField (AESC)
Code: https://github.com/giridhari-code/AERIS_64

This document is a preprint-style technical research paper for archival and citation of the architecture. It is not a claim of peer-reviewed journal acceptance.

References (selected)

Vaswani, A., et al. (2017). Attention Is All You Need. NeurIPS.
Classical and modern fast-weight / linear attention literature (delta rules, associative memory).
Mixture-of-Experts routing literature (top-k, load balancing).
Complementary learning systems / dual-memory perspectives in cognitive science and ML.